

PROYECTO GESTIX

Computación Visual

Presentado por:

Anderson David Morales Chila - amoralesch@unal.edu.co

Fredy Andres Rosero Cristancho - faroseroc@unal.edu.co

Luis Alfonso Pedraos Suarez - lpedraos@unal.edu.co

Sergio Alejandro Nova Pérez - snovap@unal.edu.co

Profesora:

Aura Maria Forero Pachon

Dec 11, 2025



Universidad Nacional de Colombia
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial
2025

1. Introducción

El proyecto Gestix surge como una propuesta innovadora orientada a explorar nuevas formas de interacción entre el usuario y el computador mediante el uso de gestos de la mano capturados en tiempo real. Su objetivo principal es desarrollar un sistema funcional para entornos Linux que permita ejecutar operaciones básicas del sistema operativo sin la necesidad de utilizar dispositivos de entrada tradicionales como el teclado o el mouse. En este sentido, Gestix busca aprovechar el potencial de la visión artificial y el procesamiento de imágenes para ofrecer una alternativa de control más natural, intuitiva y accesible.

En la versión actual del sistema, <https://github.com/GestIX-UNAL/gestix>, Gestix reconoce tres gestos principales, cada uno asociado a una acción concreta dentro del entorno Linux: modificar el brillo de la pantalla, ajustar el volumen del sistema y abrir el navegador Mozilla Firefox. Estos gestos fueron seleccionados por su utilidad en la interacción cotidiana del usuario con el sistema operativo y por su viabilidad en procesos de detección gestual en tiempo real.

Para alcanzar este funcionamiento, el proyecto implementa una serie de fundamentos propios de la computación visual. Entre ellos se incluyen la captura continua de frames desde la cámara del equipo, la aplicación de filtros y técnicas de preprocesado, y la segmentación de la región de interés para facilitar la identificación de la mano. El sistema utiliza la biblioteca OpenCV sobre Python como herramienta base para estas operaciones, permitiendo transformar y preparar cada imagen antes de enviar la información al módulo de reconocimiento gestual.

Adicionalmente, se incorporan fundamentos de geometría computacional para gestionar adecuadamente las coordenadas normalizadas de los *landmarks* de la mano y realizar transformaciones 2D necesarias para mapear las coordenadas obtenidas desde la cámara hacia las coordenadas lógicas utilizadas por la interfaz del sistema. A esto se suman conceptos esenciales de visión por computador, tales como la extracción de características, la detección y segmentación de elementos relevantes, el uso de descriptores y la operación robusta en tiempo real.

La cámara web actúa como el sensor principal del sistema, proporcionando retroalimentación inmediata que permite responder de manera ágil a los gestos del usuario. De esta forma, Gestix se configura como una interfaz alternativa basada en movimientos naturales de la mano, lo que abre camino al desarrollo de soluciones multimodales y a enfoques más accesibles en la interacción humano-computador.

El flujo general del pipeline del proyecto se presenta en la siguiente figura, donde se ilustra cada etapa del proceso, desde la captura de la imagen hasta la ejecución del comando final en el sistema operativo.

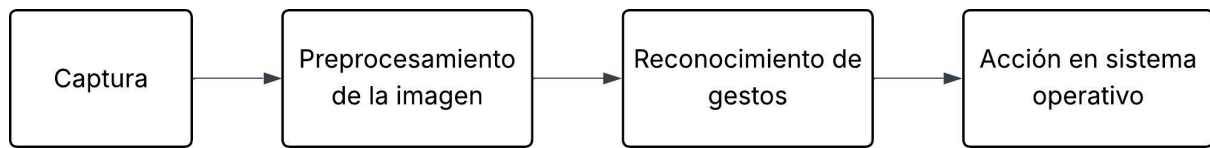


Figura 1. Pipeline del proyecto.

2. Marco Teórico

El presente marco teórico reúne los fundamentos conceptuales y tecnológicos que sustentan el desarrollo de Gestix, un sistema de interacción gestual para entornos Linux basado en visión por computador. Para ello se abordan los principios de la computación visual, la visión por computador, la geometría computacional, el reconocimiento de manos mediante *landmarks*, la interacción natural humano-computador y la ejecución de comandos del sistema operativo desde interfaces alternativas.

2.1. Computación Visual

La computación visual es un campo interdisciplinario que integra elementos de la visión por computador, el procesamiento de imágenes y la computación gráfica. Su objetivo es analizar, transformar y comprender información visual capturada por sensores como cámaras, permitiendo a los sistemas interpretar el entorno y responder en consecuencia. Dentro de esta área se emplean técnicas como la adquisición de imágenes, el preprocesamiento, la filtración, la segmentación y la extracción de características, todas fundamentales para el reconocimiento gestual (Wikipedia, 2025a; OpenCV.org, 2025).

En Gestix, la computación visual opera principalmente a través de la captura continua de fotogramas desde la cámara web y su posterior procesamiento mediante la biblioteca OpenCV, facilitando la detección de la mano y la interpretación adecuada de su configuración (Wikipedia, 2025b).

2.2. Visión por Computador

La visión por computador se centra en desarrollar algoritmos capaces de interpretar imágenes de manera similar a como lo hace el ser humano (Wikipedia, 2025). Para ello utiliza métodos de detección de bordes, segmentación de regiones, análisis de patrones, reconocimiento de objetos y seguimiento en tiempo real. En aplicaciones basadas en gestos, la visión por computador permite identificar la posición de la mano, segmentarla del fondo y estimar configuraciones específicas como la cantidad de dedos extendidos.

Gestix incorpora estos principios mediante el uso de *landmarks* y algoritmos de tracking ofrecidos por MediaPipe, cuya eficiencia permite operar en tiempo real incluso sobre hardware estándar.

2.3. Geometría Computacional y Transformaciones 2D

La geometría computacional aporta las herramientas matemáticas necesarias para el análisis y manipulación de las coordenadas obtenidas desde una imagen. Esto incluye transformaciones 2D, sistemas de referencia, coordenadas normalizadas y cálculos de distancia y orientación entre puntos (Wikipedia, 2025).

En modelos de reconocimiento de manos, los *landmarks* (Figura 2) se representan con coordenadas normalizadas en el rango $[0,1][0,1][0,1]$, lo que facilita su interpretación independientemente de la resolución de la cámara. Gestix utiliza estas coordenadas para determinar la posición relativa de cada dedo y aplicar reglas geométricas que permiten distinguir entre gestos diferentes de forma estable y robusta (Nanjundan, 2024).

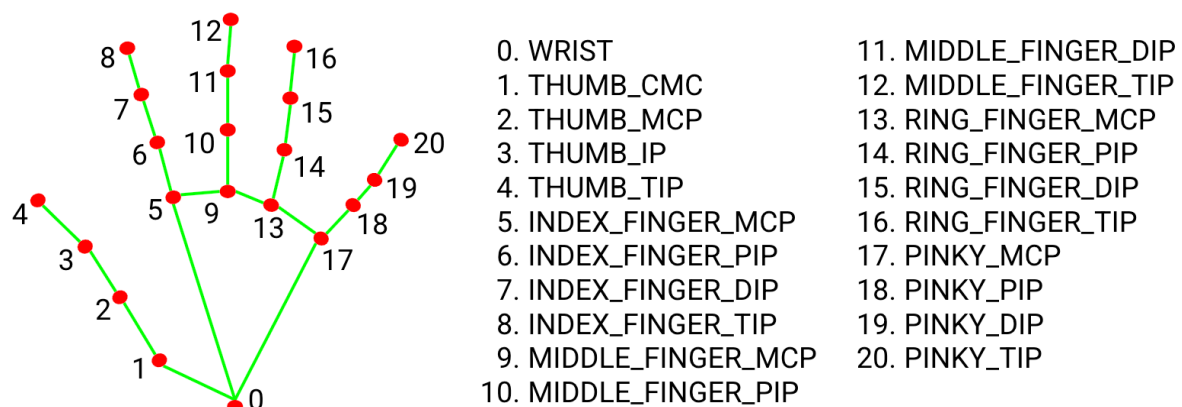


Figura 2. landmark de mano. Tomada de <https://mediapipe.readthedocs.io/en/latest/solutions/hands.html>

2.5 Detección y Seguimiento de Manos mediante Landmarks

El reconocimiento gestual moderno se apoya en modelos que identifican puntos clave de la mano para describir su estructura. MediaPipe Hands es uno de los marcos más utilizados, ya que proporciona detección y seguimiento en tiempo real mediante una red neuronal que estima 21 *landmarks* por mano (Google Research, 2025)..

Estos puntos permiten reconstruir la pose de la mano, diferenciando dedos extendidos, posiciones relativas y gestos simples o compuestos. En el caso de Gestix, la lógica del conteo de dedos se basa en la comparación entre la posición del *landmark* de cada dedo y la posición de referencia de la palma, permitiendo clasificar los gestos asociados a acciones del sistema operativo.

2.6. Interacción Humano–Computador (HCI) e Interfaces Basadas en Gestos

La interacción humano–computador estudia las formas mediante las cuales los usuarios se comunican con los sistemas informáticos. Las interfaces basadas en gestos constituyen un paradigma creciente dentro de la HCI (Vatavu, 2024), al proporcionar interacciones más naturales y similares a los movimientos cotidianos del ser humano.

Este tipo de interfaces elimina la dependencia de dispositivos físicos, permitiendo que los usuarios ejecuten comandos mediante movimientos de la mano. En Gestix, esta interacción se manifiesta en gestos específicos que sustituyen acciones tradicionales como usar teclas de volumen o mover el cursor para abrir aplicaciones. La simplicidad y diferenciación clara entre gestos contribuyen a la usabilidad y reducción de errores durante la interacción.

2.7. Integración de Sistemas y Ejecución de Comandos en Linux

Para llevar a cabo acciones como modificar el brillo o ajustar el volumen desde un sistema Linux, es habitual emplear utilidades de línea de comandos que interactúan con el servidor gráfico y con el subsistema de audio (Baeldung, 2023; Stack Exchange, 2011). Estas herramientas como `xrandr`, `brightnessctl`, `amixer`, `pactl` y `xdg-open`., pueden ser integradas en scripts o aplicaciones que encapsulan su uso para proporcionar funcionalidades de alto nivel, como las que Gestix implementa para cerrar el ciclo de interacción gestual-acción.

Gestix encapsula estas herramientas en una capa core encargada de ejecutar los comandos correspondientes una vez que el gesto ha sido reconocido. Esta integración permite cerrar el ciclo completo de la interacción gestual: percepción visual, interpretación semántica del gesto y ejecución de una acción concreta en el sistema operativo.

2.8. Procesamiento en Tiempo Real

El procesamiento en tiempo real es esencial en aplicaciones basadas en visión artificial, donde la latencia debe mantenerse baja para asegurar una experiencia fluida. Esto implica optimización en la captura de imágenes, eficiencia en los modelos de detección y velocidad en la toma de decisiones.

MediaPipe es especialmente adecuado para este tipo de aplicaciones debido a su bajo costo computacional y a su capacidad de ejecutar inferencias rápidas sin necesidad de GPU (Zhang et al., 2020). En combinación con OpenCV y el hardware de una cámara convencional, permite que Gestix responda inmediatamente a los movimientos de la mano del usuario.

3. Metodología

La metodología adoptada para el desarrollo de Gestix se estructuró en una serie de etapas progresivas que permitieron avanzar desde la definición conceptual del sistema, hasta su validación práctica en un entorno Linux real. Cada fase se diseñó con el propósito de garantizar la estabilidad, precisión y funcionalidad del sistema de interacción gestual. A continuación, se describen las etapas principales del proceso.

3.1. Definición del alcance y requerimientos funcionales

En la fase inicial se estableció el alcance del proyecto y se precisaron los requerimientos funcionales esenciales. Con el fin de mantener un sistema manejable y confiable, se optó por limitar el conjunto de gestos a un número reducido, priorizando la robustez del reconocimiento sobre la cantidad de funcionalidades. Para esta iteración del sistema se seleccionaron tres operaciones comunes que se ejecutarían en un entorno de Linux: control del brillo de la pantalla, control del volumen del sistema y apertura del navegador Mozilla Firefox, todas ellas orientadas a mejorar la interacción del usuario en un entorno Linux.

3.2. Diseño de la arquitectura del sistema

El sistema se estructuró mediante una arquitectura modular que facilita su comprensión, mantenimiento y escalabilidad. Esta arquitectura se compone de tres capas principales:

- **Capa de captura y detección de manos:** comprende la cámara web y MediaPipe Hands, responsables de obtener los frames de vídeo y detectar los *landmarks* de la mano en tiempo real.
- **Capa de controladores de alto nivel:** interpreta los gestos detectados, define las reglas de reconocimiento y asigna cada gesto a una acción específica del sistema.
- **Capa core de interacción con el sistema operativo:** ejecuta los comandos propios de Linux para modificar el brillo, ajustar el volumen o abrir aplicaciones.

Esta separación permite desacoplar la lógica de visión artificial de los procesos relacionados con la interacción directa con el sistema operativo, mejorando la legibilidad del sistema y facilitando futuras extensiones.

3.3. Implementación de la detección de manos y conteo de dedos

Para la detección gestual se empleó *MediaPipe Hands*, configurado para capturar vídeo en tiempo real y extraer los puntos clave de la mano mediante *landmarks*. A partir de estos puntos se implementó un algoritmo de conteo de dedos basado en la posición relativa entre cada dedo y la palma. Este conteo constituye la base lógica para el reconocimiento de los tres gestos definidos, permitiendo distinguir entre ellos de manera fácil y eficiente.

3.4. Diseño e implementación de los gestos de control

Se estableció un conjunto de gestos simples y fácilmente diferenciables:

- **Gesto 1:** activar la acción de ajustar el brillo.

- **Gesto 2:** activar la acción de ajustar el volumen.
- **Gesto 3:** abrir el navegador Mozilla Firefox.

Para la detección de los gestos, el sistema utiliza los landmarks proporcionados por MediaPipe Hands, los cuales representan puntos clave de la mano con coordenadas normalizadas en el rango [0,1]. Un dedo se considera extendido cuando la coordenada vertical del landmark (Figura 2) correspondiente a la punta del dedo es menor que la coordenada del landmark de la articulación proximal asociada, bajo un umbral definido empíricamente.

La clasificación de cada gesto se realiza a partir de la combinación específica de dedos extendidos, evaluando su estabilidad durante varios frames consecutivos con el fin de reducir falsos positivos causados por movimientos rápidos o ruido visual. Solo cuando un gesto se mantiene estable durante un intervalo corto de tiempo, el sistema valida su reconocimiento y procede a ejecutar la acción correspondiente en el sistema operativo.

3.5. Mapeo de gestos a comandos del sistema operativo

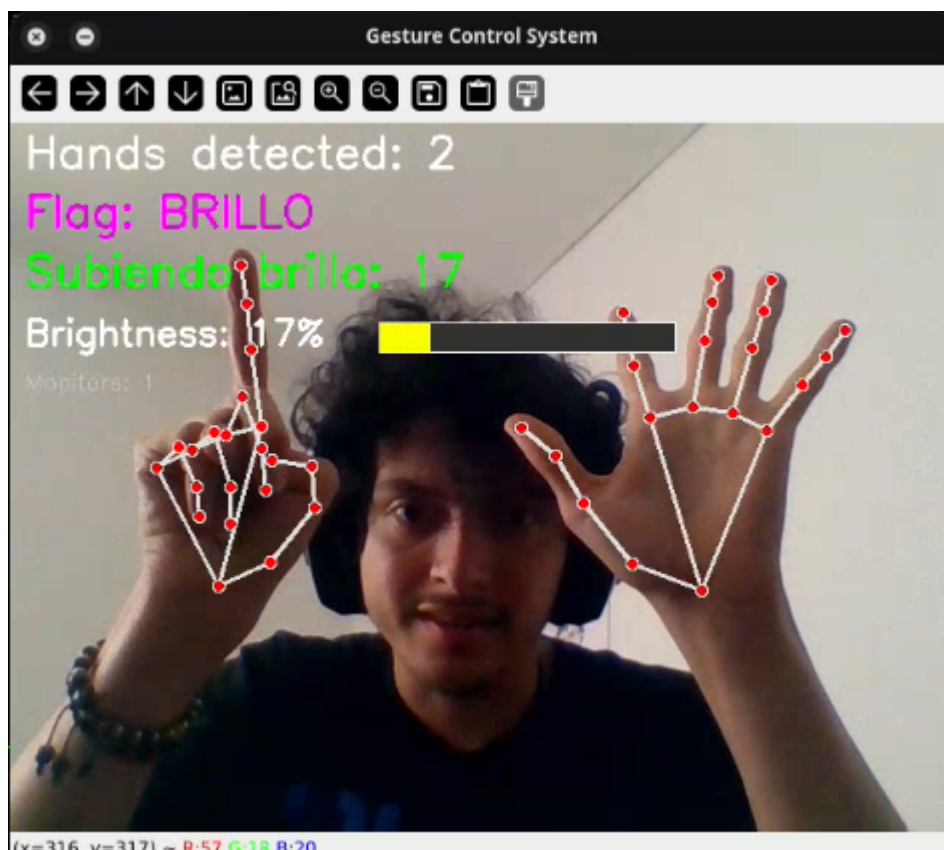


Figura 3. Ajuste de brillo por gesto de mano.

Tras el reconocimiento del gesto, la capa core del sistema invoca la acción asociada mediante herramientas propias del entorno Linux:

- Para **el brillo**, se implementaron funciones que encapsulan el uso de bibliotecas y utilidades de control de iluminación (Figura 3).
- Para **el volumen**, se realizó una abstracción sobre herramientas como pactl o amixer, permitiendo modificar el nivel de audio sin dependencia del frontend concreto.
- Para **abrir Firefox**, se empleó un comando del tipo xdg-open u otra instrucción equivalente, asegurando compatibilidad con diversos entornos de escritorio.

Este diseño mantiene el sistema flexible y portable dentro del ecosistema Linux.

3.6. Pruebas y ajustes iterativos

Finalmente, se llevó a cabo un proceso iterativo de pruebas en un entorno Linux real. Se evaluó el rendimiento del reconocimiento gestual bajo distintas condiciones ambientales, incluyendo variaciones de iluminación, distancia a la cámara y velocidad del movimiento de la mano. A partir de estos resultados se ajustaron los umbrales de detección, la sensibilidad del conteo de dedos y las reglas de decisión para minimizar falsos positivos y mejorar la experiencia general de uso. Este ciclo de pruebas permitió refinar el sistema y garantizar un funcionamiento más estable y coherente con los objetivos del proyecto.

4. Resultados

Como resultado de la implementación, se obtuvo un sistema funcional capaz de reconocer en tiempo real tres gestos de la mano y traducirlos en acciones directas sobre el sistema operativo Linux:

- Al realizar el gesto asignado al brillo, el sistema incrementa el nivel de brillo de la pantalla de forma progresiva, permitiendo al usuario ajustar la iluminación sin necesidad de utilizar el teclado o la interfaz gráfica tradicional.
- Al ejecutar el gesto asociado al volumen, el sistema incrementa el nivel de volumen del dispositivo, facilitando un control rápido del audio únicamente mediante la cámara y los gestos.
- Al realizar el gesto definido para aplicaciones, el sistema lanza el navegador Mozilla Firefox, demostrando la capacidad del sistema para invocar aplicaciones de usuario a partir de gestos predefinidos.

En las pruebas realizadas por el equipo, los tres gestos se reconocieron de manera consistente en condiciones de iluminación normales y con una distancia típica de uso frente a la cámara. El tiempo de respuesta percibido fue adecuado para la interacción en tiempo real, de forma

que el usuario observa la respuesta (subida de brillo, aumento de volumen o apertura del navegador) casi inmediatamente después de ejecutar el gesto.

Desde el punto de vista académico, el proyecto permitió validar en un caso real la combinación de visión por computadora, reconocimiento de patrones en tiempo real y control del sistema operativo, cumpliendo con los objetivos planteados inicialmente en cuanto a control de brillo, volumen y lanzamiento de aplicaciones mediante gestos. Como trabajo futuro, se proyecta extender el sistema hacia el conjunto completo de 6–8 gestos definidos en el planteamiento original, incorporar mecanismos de configuración de gestos por parte del usuario y mejorar la robustez frente a variaciones más extremas de iluminación y posición de la cámara.

5. Trabajo Futuro y Posibles Evoluciones del Proyecto

El proyecto Gestix tiene un alcance inicial centrado en el reconocimiento gestual estático y la ejecución de acciones básicas sobre un entorno Linux de escritorio. Sin embargo, existen múltiples direcciones en las cuales este trabajo puede evolucionar y expandirse en términos de funcionalidad, robustez, usabilidad e integración avanzada. A continuación se describen líneas futuras relevantes:

5.1. Reconocimiento de Gestos Dinámicos y Secuenciales

Una extensión natural de Gestix consiste en incorporar gestos dinámicos, definidos por trayectorias y cambios de configuración de la mano a lo largo del tiempo. En contraste con gestos estáticos, los gestos dinámicos (por ejemplo, barridos o curvas específicas) abren la posibilidad de comandos más complejos y expresivos. Este enfoque requeriría la integración de modelos temporales (por ejemplo, HMM o redes neuronales recurrentes) que analicen secuencias de landmarks en lugar de configuraciones individuales.

5.2. Personalización de Gestos y Aprendizaje del Usuario

Permitir que cada usuario defina y entrene gestos personalizados puede incrementar notablemente la accesibilidad y ergonomía del sistema. Esto implicaría diseñar una interfaz de entrenamiento interactivo y un mecanismo de aprendizaje incremental que almacene y reconozca las preferencias gestuales propias sin degradar el rendimiento en tiempo real.

5.3. Integración con Herramientas de Accesibilidad

Gestix puede evolucionar para integrarse con sistemas de accesibilidad existentes (como Orca o Speech Dispatcher), permitiendo que usuarios con movilidad reducida combinen gestos con otras modalidades de entrada. Además, podría habilitarse soporte para dispositivos alternativos (sensores de profundidad o cámaras 3D) para mejorar la detección en condiciones difíciles de iluminación o ángulos no estándar.

5.4. Soporte Multi-Plataforma y Portabilidad

Si bien la primera versión se centra en Linux, una ruta de evolución importante es desarrollar versiones compatibles con otros sistemas operativos como Windows y macOS. Esto implicaría abstraer las llamadas específicas del sistema operativo para gestión de volumen, brillo o ejecución de aplicaciones en una capa multiplataforma, mediante bibliotecas como Electron, Qt o plataformas de scripting multiplataforma.

5.5. Optimización de Rendimiento y Reducción de Latencia

La eficiencia en tiempo real es crítica para una experiencia fluida. En futuras iteraciones, Gestix puede integrar mecanismos de perfilado y optimización que reduzcan la latencia end-to-end (desde la captura de la imagen hasta la ejecución de la acción). Esto incluye el uso de modelos livianos, procesamiento en GPU o aceleradores de inferencia cuando estén disponibles, y algoritmos de filtrado temporal para mejorar la estabilidad sin sacrificar velocidad.

5.6. Evaluación en Contextos No Controlados

Hasta ahora, las pruebas experimentales se han llevado a cabo en condiciones relativamente controladas. Una evolución significativa sería evaluar y ajustar el sistema en escenarios más diversos, incluyendo variaciones de luz extrema, fondos complejos y múltiples usuarios simultáneos. Esto permitiría medir la robustez real del sistema y guiar mejoras basadas en métricas objetivas.

5.7. Expansión de Acciones y Mapeos Complejos

Además de las acciones básicas de brillo, volumen o apertura de aplicaciones, Gestix podría mapear gestos a comandos más complejos, como secuencias de atajos de teclado, macros automatizados, o interacciones con aplicaciones específicas (por ejemplo, control de presentaciones, edición de video, o navegación web sin contacto físico).

En conjunto, estas líneas de trabajo no solo incrementan la utilidad del sistema en entornos reales, sino que también convierten a Gestix en una base sólida para investigación adicional en interacción natural, visión por computador y accesibilidad. Cada una de estas extensiones puede constituir un objetivo de trabajo independiente, contribuyendo a madurar el proyecto hacia una solución más general, robusta y adaptada a las necesidades de distintos tipos de usuarios.

6. Bibliografía

- Wikipedia. (2025a). *Visión artificial*. En Wikipedia, la enciclopedia libre. https://es.wikipedia.org/wiki/Visi%C3%B3n_artificial
- OpenCV.org. (2025). *OpenCV – Open Computer Vision Library*. <https://opencv.org/>
- Wikipedia. (2025b). *OpenCV*. En Wikipedia, la enciclopedia libre. <https://es.wikipedia.org/wiki/OpenCV>

- Wikipedia. (2025). *Geometría computacional*. En *Wikipedia, la enciclopedia libre*. Recuperado de https://es.wikipedia.org/wiki/Geometr%C3%ADa_computacional Wikipedia
- Google Research. (2025). *MediaPipe Hands: On-device real-time hand tracking*. Recuperado de https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
- Vatavu, R.-D. (2024). *Gesture-based interaction*. En C. Stephanidis & G. Salvendy (Eds.), *Human-Computer Interaction: Foundations, Advances, Methods, Technologies, and Applications* (Vol. 4, pp. 155–178). CRC Press. <https://www.routledge.com/Interaction-Techniques-and-Technologies-in-Human-Computer-Interaction/Stephanidis-Salvendy/p/book/9781032370033>
- Fiveable Content Team. (2025). *Geometric transformations in computer vision and image processing*. Recuperado de <https://fiveable.me/computer-vision-and-image-processing/unit-2/geometric-transformations/study-guide/ABJtpzXnr3vbdF69> Fiveable
- Zhang, F., Bazarevsky, V., Vakunov, A., et al. (2020). *MediaPipe Hands: On-device real-time hand tracking*. Retrieved from <https://arxiv.org/abs/2006.10214>